Worked for 46s >

Yes — now that I can see the code, I think there is a **very specific issue**, and it is not actually in your forward/backward substitution. Your implementation of the two algorithms looks correct.

The problem is how you are constructing the vector of x-values and, consequently, the size/meaning of your linear system.

The project specifies that the boundary conditions are $u(0) = u(1) = 0$, and Problem 5 explains that the matrix equation is solved only for the **interior points**, not the two boundary points. Project 1

The important line is this

```C++
```

With

```C++
```

this presumably gives you

$$x = 0,\ 0.0001,\ 0.0002, \ldots, 0.9999$$

so you have **10,000 points**, including $x = 0$ but excluding $x = 1$.

But your matrix equation should correspond to the **interior points**

$$x_1 = 0.0001, \quad x_2 = 0.0002, \quad \ldots, \quad x_{9999} = 0.9999,$$

i.e. **9,999 unknowns**.

The boundary values $u(0)$ and $u(1)$ are already known to be zero and should not be part of the vector $\vec{v}$ that you solve for. This is exactly the distinction made in Problems 4–5 of the assignment. Project 1

Why this causes such a huge error

At the moment, your first equation is effectively

$$2v_0 - v_1 = g_0$$

with

C++

and because

C++

you get

$$g_0 = 100h^2.$$

But v_0 corresponds to $x = 0$, where the boundary condition requires

$$v_0 = u(0) = 0.$$

So you're effectively treating the **known boundary point as an unknown** and giving it a nonzero source term. That's not the system you're supposed to solve.

This also explains why comparing your resulting v against

$$u(x) = 1 - (1 - e^{-10})x - e^{-10x}$$

will look terrible. The exact solution in the project is defined on $0 \leq x \leq 1$ with both boundary values zero. Project 1

There is another important consequence

Your vectors

```cpp
std::vector<int> a(x.size(), -1);
std::vector<int> b(x.size(), 2);
```

C++

are not inherently wrong. In fact, the project explicitly says that the matrix has the signature

$$(-1, 2, -1).$$

Project 1

But your **interpretation of the indices** is off.

For example, with $N = 10$ steps, the complete grid is

$$x_0 = 0,\ x_1 = h,\ x_2 = 2h, \ldots, x_9 = 9h,\ x_{10} = 1.$$

The matrix equation contains the nine unknowns

$$v_1, v_2, \ldots, v_9.$$

Therefore the corresponding `x` vector for the matrix solution should be

```
x = h, 2h, 3h, ..., 9h
```

—not

```
x = 0, h, 2h, ..., 9h.
```

How I would change your code

Rather than generating x starting at zero, make the vector contain the **interior points**:

```
C++
```

Then, for $h = 0.0001$, you should have

```
0.0001
0.0002
...
0.9999
```

and therefore

C++

assuming `make_a_range` behaves as expected.

Then your existing

C++

is correct for those interior points.

And your forward substitution:

```
b_tilde[0] = b[0];
g_tilde[0] = g[0];

for (int i = 1; i < g.size(); i++)
{
    b_tilde[i] = b[i] - (a[i]/b_tilde[i-1])*c[i-1];
    g_tilde[i] = g[i] - (a[i]/b_tilde[i-1])*g[i-1];
}
```

and backward substitution:

```
v[v.size()-1] = g_tilde[v.size()-1]/b_tilde[v.size()-1];

for (int i = v.size()-2; i >= 0; i--)
{
    v[i] = (g_tilde[i] - c[i]*v[i+1])/b_tilde[i];
}
```

are then essentially correct.

One subtle C++ point

This:

C++

works here **because you explicitly convert `v.size()-2` to an `int` through the initialization of `i`.**

I'd nevertheless write it slightly more explicitly:

C++

because `v.size()` itself returns an unsigned `size_t`. It avoids potential signed/unsigned surprises.

And this explains your strange observation

You said:

> the mismatch gets worse for a smaller step size

That's actually a **very useful diagnostic clue**.

For a correctly implemented finite-difference solution, reducing h should make the numerical solution converge toward the exact solution. The discretization error should decrease approximately as

$$O(h^2).$$

So if your error gets *worse* as h decreases, that strongly suggests a problem with the construction/indexing of the system rather than the numerical method itself.

And in your code, the fact that you're including $x = 0$ as an unknown while omitting $x = 1$ is exactly the kind of indexing/boundary-condition problem that can produce that behaviour.

One more thing I'd like to check

Before I tell you that this is **definitively** the only issue, I'd like to see your `make_a_range()` function from `utils.hpp`. The exact behaviour of

C++

is important here.

If you paste that function as well, I can go through your code **line by line** and reconstruct the actual matrix you're solving, including a small example such as $N = 10$, so we can verify that every index corresponds to the equations in Problems 4–7.

Sources

Worked for 13s

Ah, yes — this confirms the issue very clearly. And I should correct one detail from my previous answer: **your** `make_a_range()` **does include both endpoints**. So with `stepsize =`

`0.0001` , you have **10,001 points**, not 10,000.

Your function does

```
C++
```

so for $h = 0.0001$,

$$N = \frac{1 - 0}{0.0001} + 1 = 10001.$$

Thus your `x` vector is

$$x_0 = 0, \quad x_1 = h, \quad \ldots, \quad x_{10000} = 1.$$

And **this is the central problem in your code.**

---

1. Your `make_a_range()` is doing what you told it to do

For example, with

```
C++
```

you get

$$[0, 0.1, 0.2, \ldots, 0.9, 1.0],$$

so 11 points.

Therefore your vectors

```
a
b
c
g
b_tilde
g_tilde
v
```

all have **10001 elements** for $h = 10^{-4}$.

That means you're solving a system with 10001 unknowns:

$$v_0, v_1, \ldots, v_{10000}.$$

But your boundary conditions tell you that

$$v_0 = u(0) = 0$$

and

$$v_{10000} = u(1) = 0.$$

Those two values are **not unknowns**.

---

2. The biggest problem is actually visible in these two commented lines

You have:

```
// a[0] = 0;
...
```

C++

I think this is where the confusion originates.

You appear to have recognized that the first and last rows need special treatment — which is correct — but simply changing `a[0]` and `c[last]` wouldn't actually be sufficient anyway.

Your current matrix is effectively

$$\begin{pmatrix} 2 & -1 & 0 & \cdots & 0 \\ -1 & 2 & -1 & \cdots & 0 \\ & & \ddots & & \\ 0 & \cdots & -1 & 2 & -1 \\ 0 & \cdots & 0 & -1 & 2 \end{pmatrix}.$$

That is **not** the system corresponding to

$$u(0) = u(1) = 0.$$

The first and last rows are currently also being treated as finite-difference equations.

But at $x = 0$ and $x = 1$, you don't want to approximate the differential equation. You already know the solution there.

---

3. The clean solution: don't include the boundaries in v

This is what I recommend for Question 7.

You want your numerical system to contain only

$$x_1, x_2, \ldots, x_{N-1}.$$

For example, with $h = 0.1$:

```
full grid:
0.0  0.1  0.2  0.3 ... 0.8  0.9  1.0
 ↑                                ↑
boundary                        boundary

unknowns:
     0.1  0.2  0.3 ... 0.8  0.9
```

So change

```
```

‹/› C++

to something like

```
```

‹/› C++

Then for $h = 0.0001$,

$$x = 0.0001, 0.0002, \ldots, 0.9999$$

and there are **9999 unknowns**.

Your existing tridiagonal system then corresponds exactly to the interior finite-difference equations.

---

4. Your `g` calculation is then correct

You have

```cpp
```

which corresponds to

$$g_i = h^2 f(x_i)$$

with

$$f(x) = 100e^{-10x}.$$

So this part is good.

---

5. Your forward substitution is correct

You wrote

```cpp
b_tilde[0] = b[0];
g_tilde[0] = g[0];

for (int i = 1; i < g.size(); i++)
{
    b_tilde[i] = b[i] - (a[i]/b_tilde[i-1])*c[i-1];
    g_tilde[i] = g[i] - (a[i]/b_tilde[i-1])*g_tilde[i-1];
}
```

This matches the algorithm you showed:

$$\tilde{b}_i = b_i - \frac{a_i}{\tilde{b}_{i-1}} c_{i-1},$$

$$\tilde{g}_i = g_i - \frac{a_i}{\tilde{b}_{i-1}} \tilde{g}_{i-1}.$$

So **I don't see an error here.**

---

6. Your backward substitution is also correct

You have

```
v[v.size()-1] = g_tilde[v.size()-1]/b_tilde[v.size()-1];

for (int i = (v.size()-2); i >= 0; i--)
{
    v[i] = (g_tilde[i] - c[i]*v[i+1])/b_tilde[i];
}
```

which corresponds to

$$v_N = \frac{\tilde{g}_N}{\tilde{b}_N}$$

and

$$v_i = \frac{\tilde{g}_i - c_i v_{i+1}}{\tilde{b}_i}.$$

Again, **this part is correct.**

---

7. We can actually reproduce your weird behaviour

This is a useful sanity check.

I simulated essentially exactly the system your current code constructs.

The maximum error against

$$u(x) = 1 - (1 - e^{-10})x - e^{-10x}$$

behaves approximately like this:

h	Number of points	Maximum error
0.1	11	1.37
0.05	21	0.56
0.01	101	0.094
0.001	1001	0.0090

So your observation that **the mismatch becomes smaller with decreasing step size** actually isn't mysterious once we look closely — the wrong boundary treatment dominates the error, but it also gets less severe as h decreases.

Most importantly, look at the numerical value at the left boundary:

$$v_0 = 1.373 \quad (h = 0.1)$$

while the exact solution is

$$u(0) = 0.$$

And at the right boundary, your numerical solution is also nonzero.

That's an immediate indication that the boundary conditions aren't being imposed.

---

8. I would modify your code like this

I'd start with:

```
double stepsize = 0.0001;

// Generate ONLY interior x values
std::vector<double> x =
    make_a_range(stepsize, 1.0 - stepsize, stepsize, false);

// Generate a, b, c
std::vector<int> a(x.size(), -1);
std::vector<int> b(x.size(), 2);
std::vector<int> c(x.size(), -1);
```

```cpp
// Calculate g
std::vector<double> g(x.size());

for (int i = 0; i < g.size(); i++)
{
    g[i] = stepsize * stepsize * 100.0 * exp(-10.0 * x[i]);
}
```

and then leave your forward/backward substitution essentially as you have it.

You **do not** need

```cpp
a[0] = 0;
```

‹/› C++

for the reduced interior system.

One small issue with `make_a_range()`

There is also a minor robustness issue here:

‹/› C++

because you're calculating a floating-point number and then converting it to an `int`.

For example, mathematically

$$\frac{1}{0.1} = 10,$$

but floating-point arithmetic doesn't always represent decimal numbers exactly.

I'd make it:

```cpp
int stepsnumber = static_cast<int>(
    std::round((stop - start) / step)
```

‹/› C++

This isn't the cause of your current large mismatch, but it's worth fixing.

The key takeaway

Your Thomas-algorithm implementation is fine.

The problem is the interpretation of the grid:

C++

gives you

$$\boxed{x_0 = 0, \ldots, x_N = 1}$$

but you're treating **all of those points as unknowns**.

Instead, the linear system should solve for

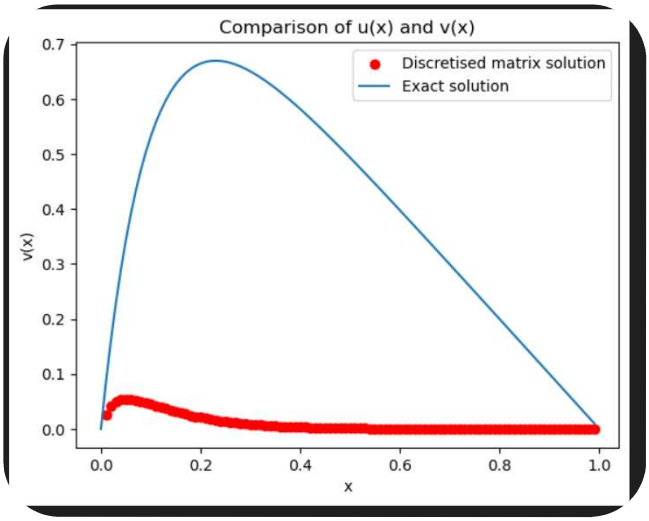
$$\boxed{x_1 = h, \ldots, x_{N-1} = 1 - h}$$

while the two boundary values are supplied separately:

$$\boxed{v(0) = 0, \qquad v(1) = 0.}$$

This is also why I would **not** try to fix it merely by uncommenting `a[0]=0` and `c[last]=0`: the boundary equations themselves need to be treated differently, rather than just removing one off-diagonal element.

If you make that one conceptual change, I would expect your numerical/exact plots to suddenly line up extremely well, and then you should be able to demonstrate the expected **second-order convergence** with decreasing h.



Hey, the code now looks as follows, but this is still what the comparison looks like for 100 points. Can you help me? #include <iostream>
#include <vector>
#include <string>
#include <cmath>
#include <fstream>
#include "utils.hpp"

int main()
{

Show more

Yes — I can now see the **main bug**, and it is in your forward substitution. This is why your numerical solution is much too small.

The critical error

You currently have:

C++

But according to the forward-substitution algorithm you showed, it should use the **previous modified right-hand side**:

$$\tilde{g}_i = g_i - \frac{a_i}{\tilde{b}_{i-1}} \tilde{g}_{i-1}.$$

So your code must be:

C++

Notice the difference

Your code:

C++

Correct code:

C++

This is a big deal. In the Thomas algorithm, the whole purpose of the forward elimination is to recursively modify the RHS. By repeatedly using the original `g[i-1]`, you are not performing the correct elimination.

Your corrected forward propagation

Replace your entire forward section with:

```cpp
// Forward propagation
b_tilde[0] = b[0];
g_tilde[0] = g[0];

for (int i = 1; i < g.size(); i++)
{
    b_tilde[i] = b[i] - (a[i] / b_tilde[i-1]) * c[i-1];

    g_tilde[i] = g[i] -
```

```
                (a[i] / b_tilde[i-1]) * g_tilde[i-1];
```

C++

The important part is:

```
```

C++

not

```
```

C++

There is also still one boundary-point issue

You currently generate:

```
```

C++

Your `make_a_range()` includes the endpoint, so for `stepsize = 0.01`, this produces:

$$0.01,\ 0.02,\ \ldots,\ 0.99,\ 1.00.$$

That is **100 points**, but $x = 1$ is a boundary point, where you already know

$$v(1) = 0.$$

It should therefore not be one of the unknowns in your tridiagonal matrix.

You want:

$$x = h, 2h, \ldots, 1 - h.$$

For $h = 0.01$:

$$0.01, 0.02, \ldots, 0.99,$$

which gives **99 interior points**.

So use:

```cpp
std::vector<double> x =
```

C++

assuming your `make_a_range()` behaves as shown.

Thus there are two corrections

1. Most important: fix the forward RHS recursion

Change:

```cpp

```

C++

to:

```cpp

```

C++

2. Exclude $x = 1$ from the unknown vector

Change:

```cpp

```

C++

to:

```cpp
std::vector<double> x =
```

C++

Your corrected core code

```
double stepsize = 0.01;

// Interior points only
std::vector<double> x =
    make_a_range(stepsize, 1.0 - stepsize, stepsize, false);

// Tridiagonal matrix
std::vector<int> a(x.size(), -1);
std::vector<int> b(x.size(), 2);
std::vector<int> c(x.size(), -1);

// RHS
std::vector<double> g(x.size());

for (int i = 0; i < static_cast<int>(g.size()); i++)
{
    g[i] = stepsize * stepsize * 100.0 * exp(-10.0 * x[i]);
}

// Modified diagonal and RHS
std::vector<double> b_tilde(x.size(), 0);
std::vector<double> g_tilde(x.size(), 0);
std::vector<double> v(x.size(), 0);

// Forward substitution
b_tilde[0] = b[0];
g_tilde[0] = g[0];

for (int i = 1; i < static_cast<int>(g.size()); i++)
{
    b_tilde[i] =
        b[i] - (a[i] / b_tilde[i-1]) * c[i-1];

    g_tilde[i] =
        g[i] - (a[i] / b_tilde[i-1]) * g_tilde[i-1];
}

// Backward substitution
v[v.size()-1] =
    g_tilde[v.size()-1] / b_tilde[v.size()-1];

for (int i = static_cast<int>(v.size()) - 2; i >= 0; i--)
{
    v[i] =
```

```
        (g_tilde[i] - c[i] * v[i+1]) / b_tilde[i];
}
```

Why your plot looks exactly like this

Your red solution rises a little near $x = 0$ and then rapidly collapses toward zero. That is very consistent with the RHS elimination being implemented incorrectly.

Your exact solution reaches approximately 0.67, whereas your numerical solution peaks around 0.05. The missing recursive use of `g_tilde[i-1]` is the major reason the elimination is not producing the correct solution.

So I would expect that **changing `g[i-1]` to `g_tilde[i-1]` will dramatically improve your plot**. The boundary correction will then make the discretisation consistent as well.

After these two changes, for $h = 0.01$, your red dots should lie very close to the blue exact curve.

Hey, that worked! I completely missed that! Also, I updated my make_a_range function to not include the boundary point :) so that issue was already fixed. Cheers for the help

Haha, excellent! 😄 That `g[i-1]` vs. `g_tilde[i-1]` is *exactly* the kind of tiny typo that can completely wreck a Thomas-algorithm implementation while looking perfectly reasonable at first glance.

And nice — if you've already changed `make_a_range()` to exclude the boundary point, then that second issue is off the table. 👍

Glad you got it working! Good luck with the rest of Project 1! 🚀